

Teoria e Aplicação de Grafos

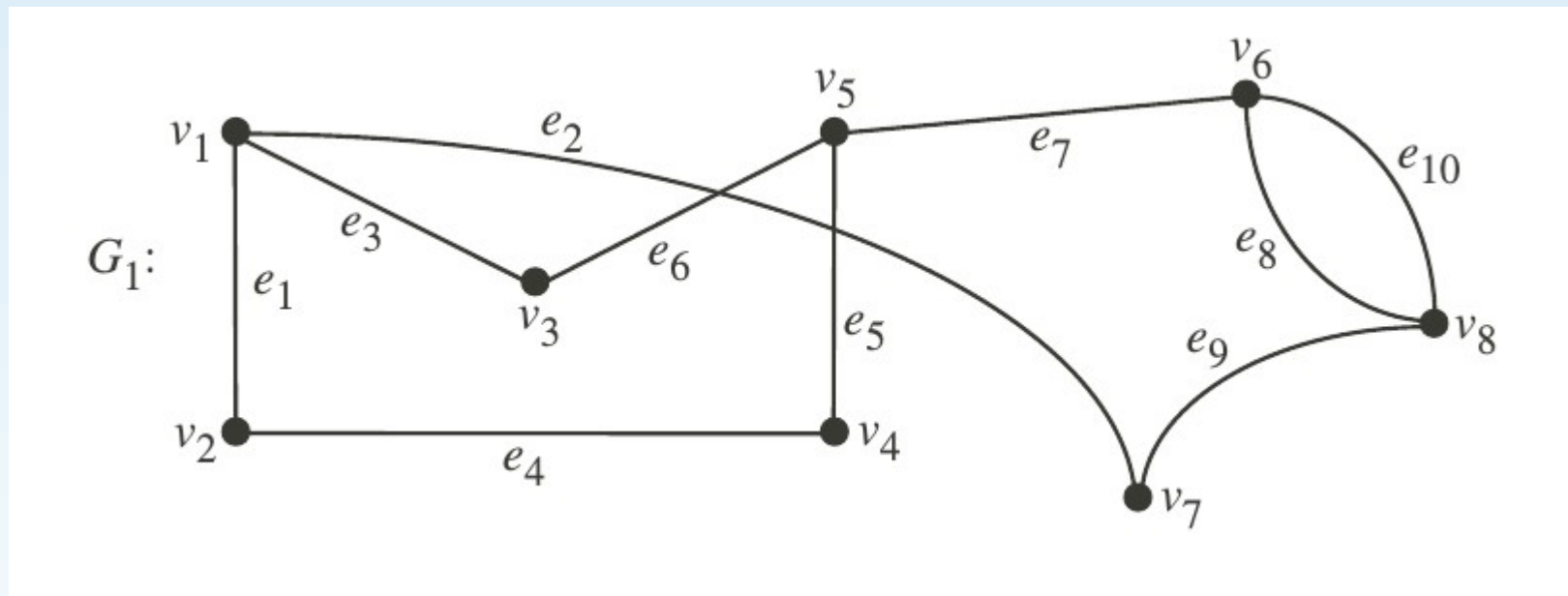
Roteiro da aula:

- **Grafos com Pesos (ou Valorados)**
- **Algoritmo de Dijkstra (Menor caminho entre dois vértices em grafos valorados)**
- **Todos os menores caminhos entre pares de vértices: Algoritmo Floyd-Warshall**
- **Percursos de Euler**
- **Percursos de Hamilton**
- **Teoremas de Ore e de Dirac**

Grafos Valorados

- Assim como as arestas (elos) de um grafo podem ser rotuladas, elas podem receber valores numéricos ou até representarem funções (com entrada e saída).

Exemplo: Grafos Valorados



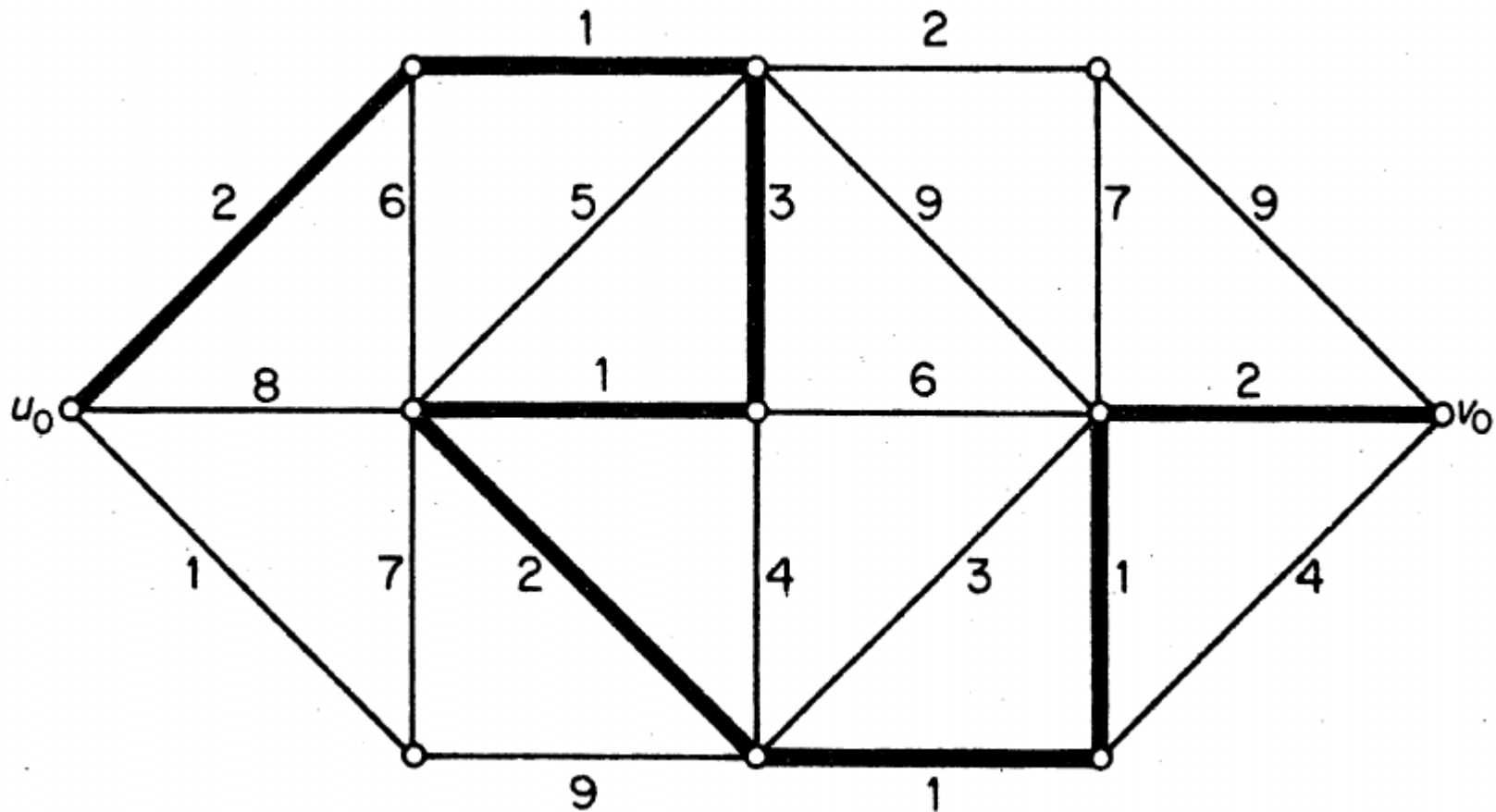
$G_1=(V,E)$

$V=\{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8\}$

$E=\{e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_8, e_9, e_{10}\}$

Ruohonen, 2013.

Exemplo: Grafos Valorados



Bondy & Murty, 1982.

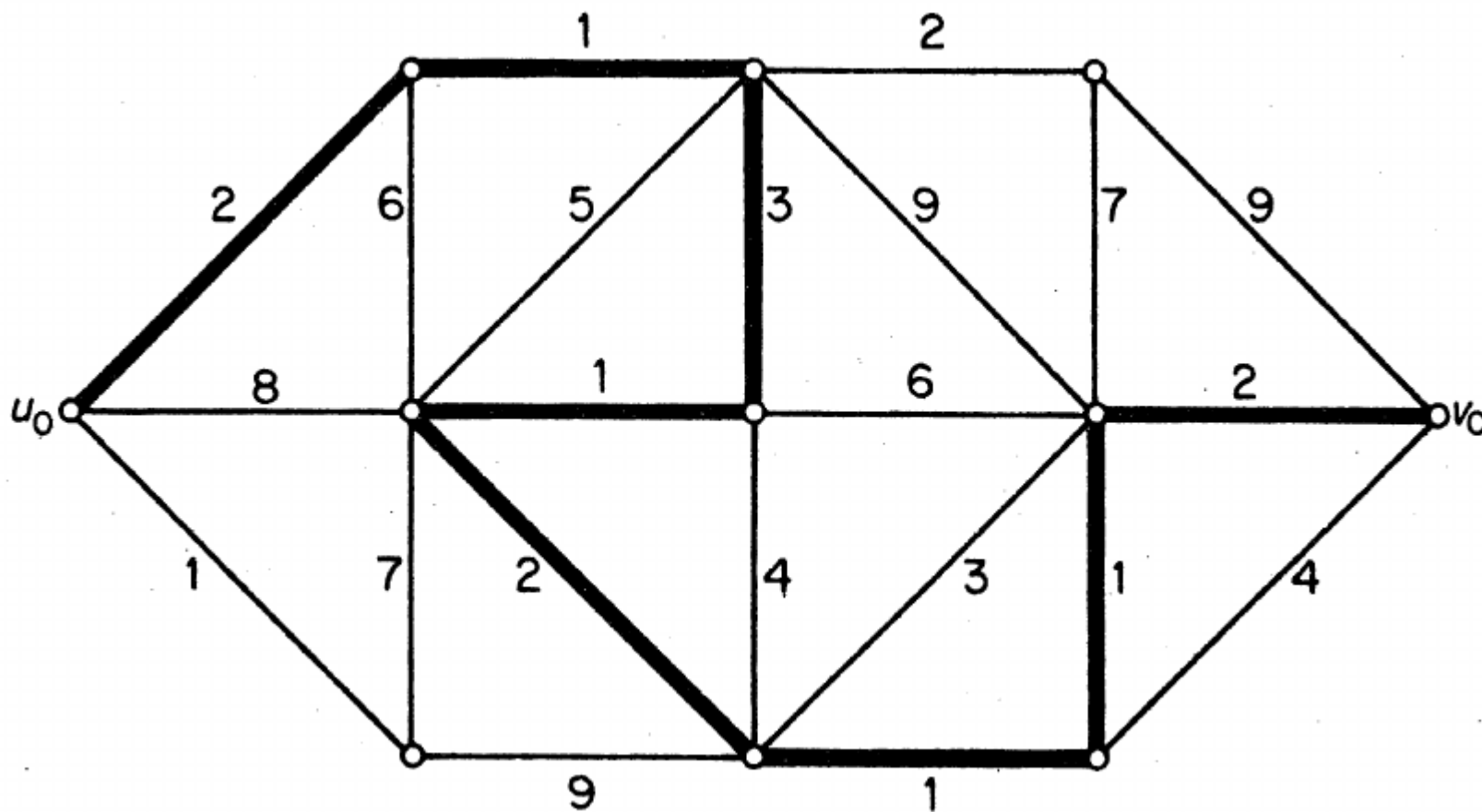
Grafos Valorados

- Em Grafos Valorados (i.e. com pesos), tipicamente são atribuídos pesos $W(E)$ aos elos (arestas) em forma de números (inteiros ou reais).

Grafos Valorados

- Em Grafos Valorados (i.e. com pesos), tipicamente são atribuídos pesos $W(E)$ aos elos (arestas) em forma de números (inteiros ou reais).
- Tal representação explicita um problema prático que é o de encontrar o **caminho mínimo(ou máximo)** entre dois vértices de um grafo, agora a soma desses pesos $W()$ das arestas constituirá o valor desse menor caminho. Típico problema de otimização.

Exemplo: Grafos Valorados



Como no exemplo, onde o menor caminho entre u_0 e v_0 está marcado em negrito.

Bondy & Murty, 1982.

Algoritmo de Dijkstra

- Publicado em 1959,
Dijkstra, E. W. (1959). "A note on two problems in connexion with graphs"
<http://www-m3.ma.tum.de/twiki/pub/MN0506/WebHome/dijkstra.pdf>. Numerische Mathematik1: 269–271.
doi:10.1007/BF01386390.

Algoritmo de Dijkstra

- Mantém um conjunto S de vértices cujos caminhos mais curtos até um vértice origem já são conhecidos.
- Produz uma árvore de caminhos mais curtos de um vértice origem s para todos os vértices que são alcançáveis a partir de s .

Ziviani, 2011.

Algoritmo de Dijkstra

- Utiliza a técnica de **relaxamento**:
 - Para cada vértice $v \in V$ o atributo $p[v]$ é um limite superior do peso de um caminho mais curto do vértice origem s até v .
 - O vetor $p[v]$ contém uma estimativa de um caminho mais curto.

Ziviani, 2011.

Algoritmo de Dijkstra

- O primeiro passo do algoritmo é inicializar os antecessores e as estimativas de caminhos mais curtos:
 - $Antecessor[v] = nil$ para todo vértice $v \in V$,
 - $p[u] = 0$, para o vértice origem s , e
 - $p[v] = \infty$ para $v \in V - \{s\}$.

Ziviani, 2011.

Algoritmo de Dijkstra

- O **relaxamento** de uma aresta (u, v) consiste em verificar se é possível melhorar o melhor caminho até v obtido até o momento se passarmos por u .
- Se isto acontecer, $p[v]$ e $Antecessor[v]$ devem ser atualizados.

```
if  $p[v] > p[u] + \text{peso da aresta } (u, v)$   
then  $p[v] = p[u] + \text{peso da aresta } (u, v)$   
     $Antecessor[v] := u$ 
```

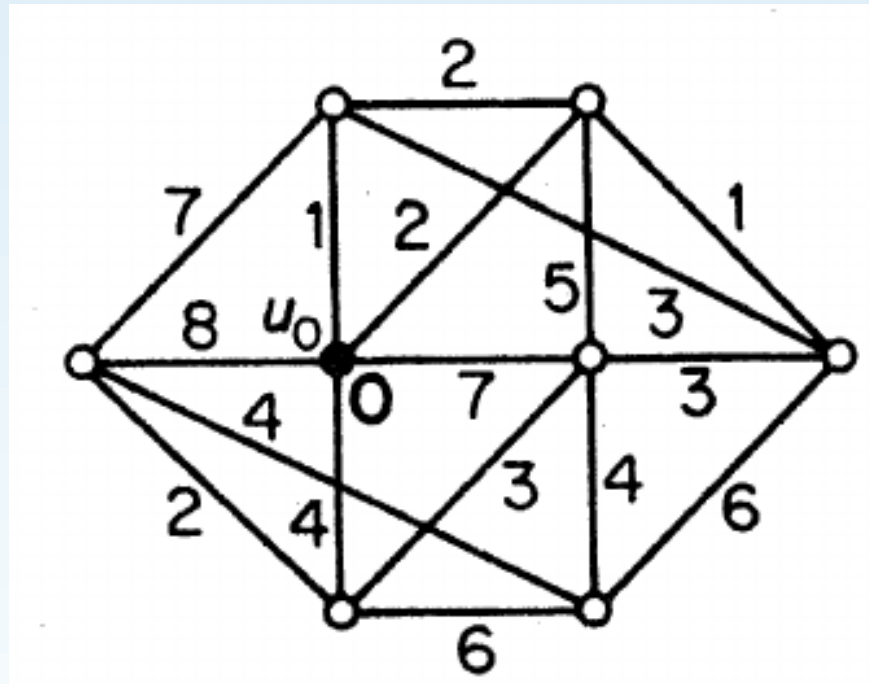
Ziviani, 2011.

Algoritmo de Dijkstra

1. $l(u_0) = 0, l(v) = \infty$ para $v \neq u_0, S_0 = \{u_0\}$ e $i = 0$
2. Para cada $v \in \bar{S}_i$,
 - trocar $l(v)$ por $\min\{l(v), l(u_i) + w(u_i v)\}$
 - Calcular $\min_{v \in \bar{S}_i}\{l(v)\}$
 - Fazer $u_{i+1} = \min_{v \in \bar{S}_i}\{l(v)\}$
 - Fazer $S_{i+1} = S_i \cup \{u_{i+1}\}$
3. Se $i = v - 1$, parar
4. Se $i < v - 1$, trocar i por $i + 1$ e ir para passo 2.

Exemplo: Algoritmo de Dijkstra

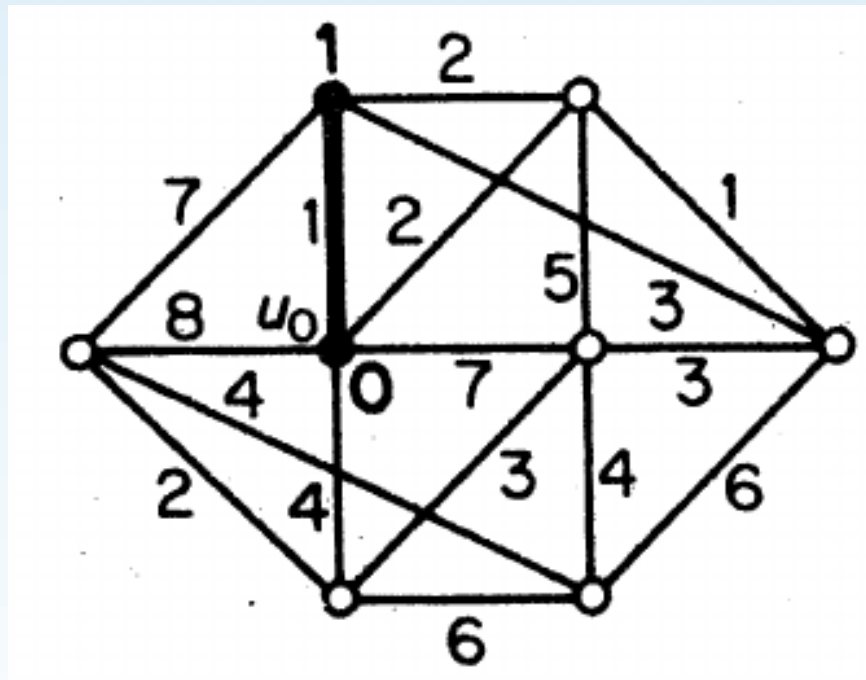
A partir de u_0 , encontrar todos os menores caminhos a todos os outros vértices
Usando Dijkstra



Bondy & Murty, 1982.

Exemplo: Algoritmo de Dijkstra

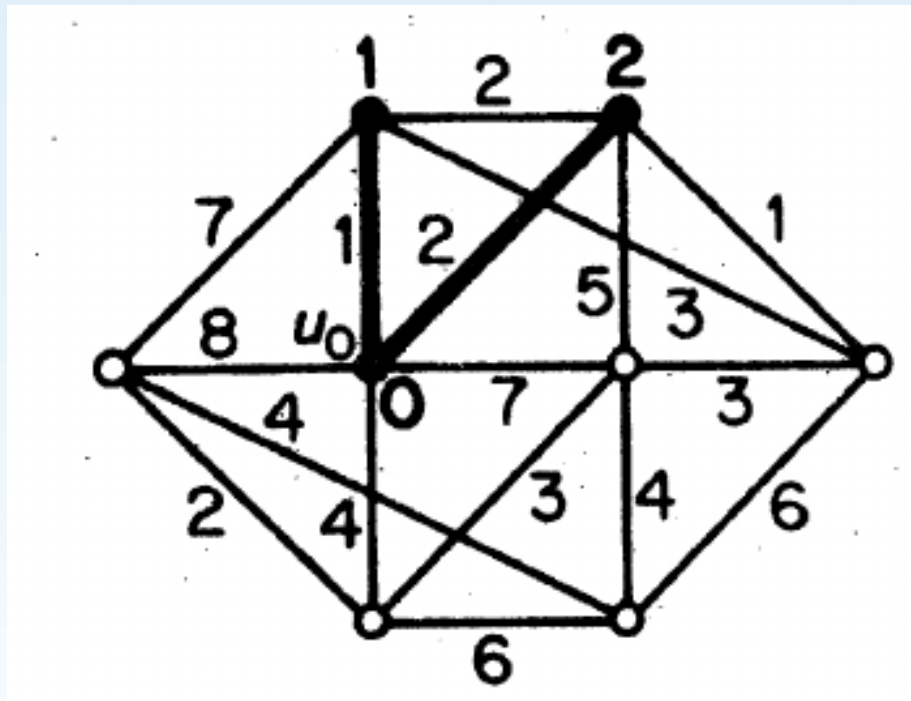
A partir de u_0 , encontrar todos os menores caminhos a todos os outros vértices
Usando Dijkstra



Bondy & Murty, 1982.

Exemplo: Algoritmo de Dijkstra

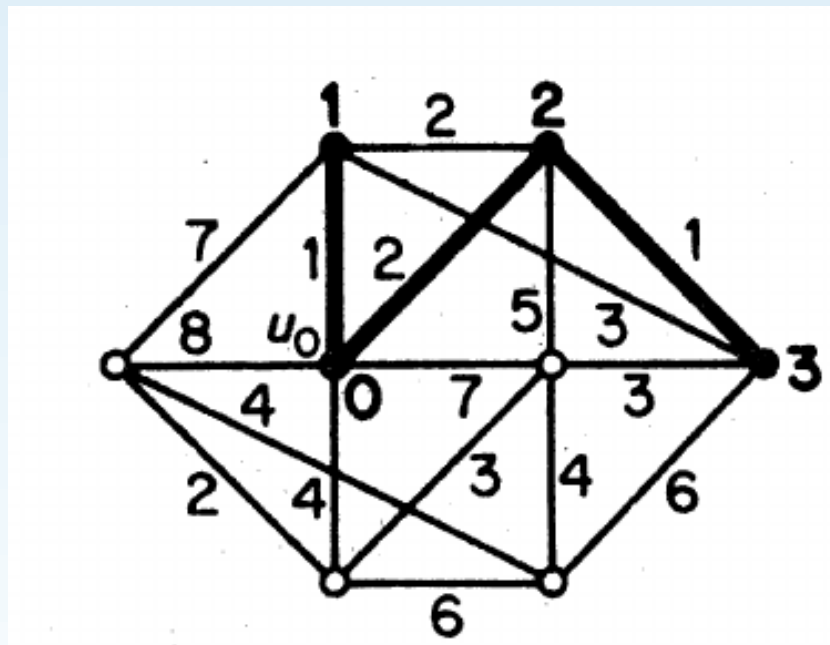
A partir de u_0 , encontrar todos os menores caminhos a todos os outros vértices
Usando Dijkstra



Bondy & Murty, 1982.

Exemplo: Algoritmo de Dijkstra

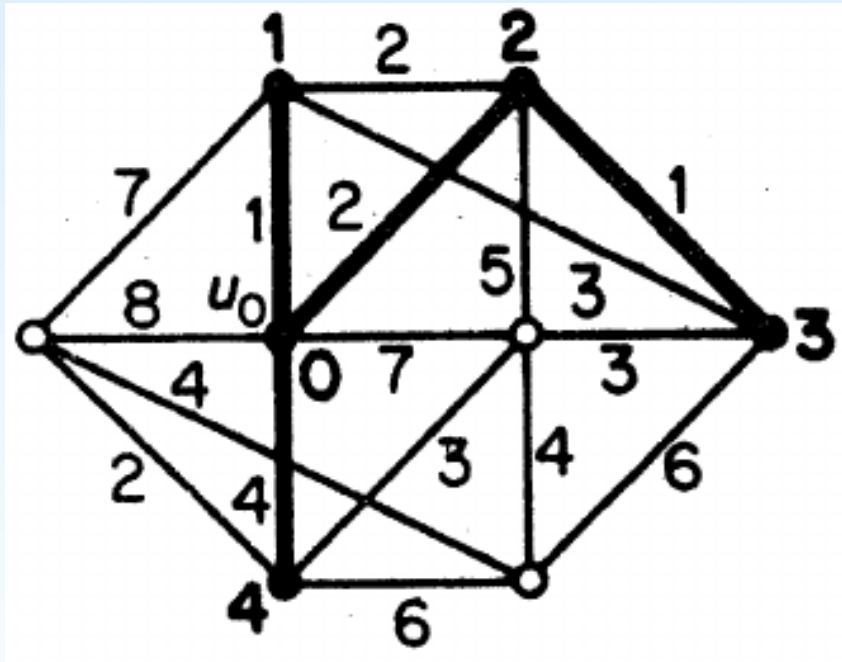
A partir de u_0 , encontrar todos os menores caminhos a todos os outros vértices
Usando Dijkstra



Bondy & Murty, 1982.

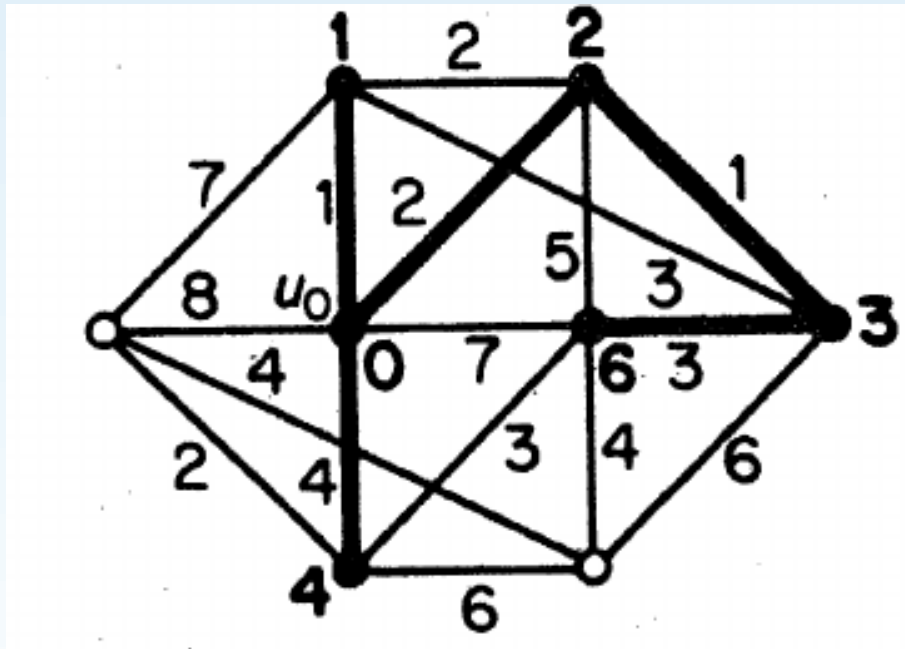
Exemplo: Algoritmo de Dijkstra

A partir de u_0 , encontrar todos os menores caminhos a todos os outros vértices
Usando Dijkstra



Exemplo: Algoritmo de Dijkstra

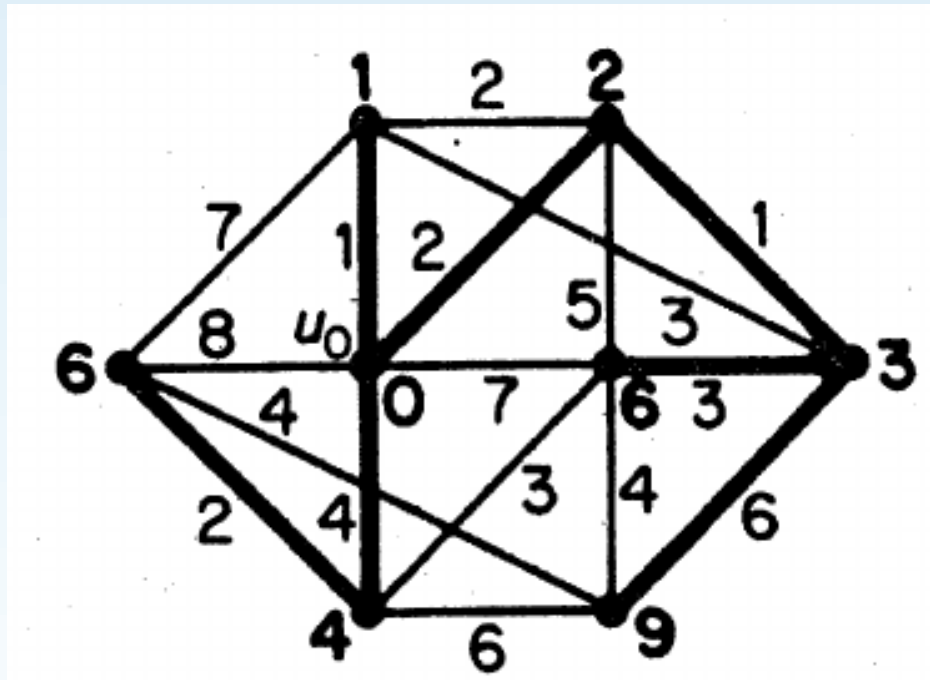
A partir de u_0 , encontrar todos os menores caminhos a todos os outros vértices
Usando Dijkstra



Bondy & Murty, 1982.

Exemplo: Algoritmo de Dijkstra

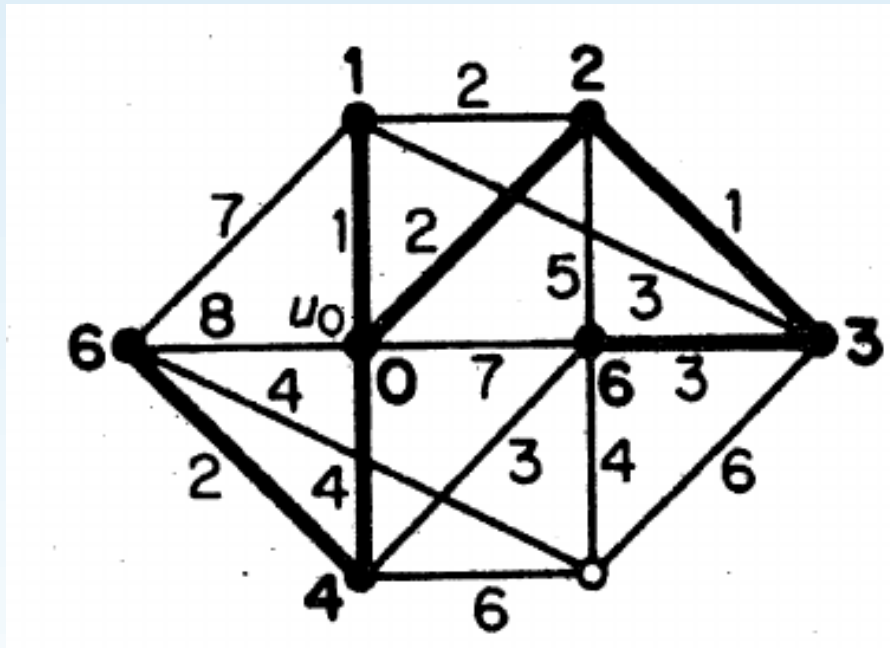
A partir de u_0 , encontrar todos os menores caminhos a todos os outros vértices
Usando Dijkstra



Bondy & Murty, 1982.

Exemplo: Algoritmo de Dijkstra

A partir de u_0 , encontrar todos os menores caminhos a todos os outros vértices
Usando Dijkstra



Cada caminho gera uma árvore com raiz em u_0

Bondy & Murty, 1982.

Considerações sobre algoritmo de Dijkstra

- Para menor caminho entre dois vértices

$$\Theta(|V|^2)$$

Considerações sobre algoritmo de Dijkstra

- Para menor caminho entre dois vértices

$$\Theta(|V|^2)$$

- Para todos os menores caminhos entre os pares de vértices

$$\Theta(|V|^3 \ln |V|)$$

Considerações sobre algoritmo de Dijkstra

- Para menor caminho entre dois vértices

$$\Theta(|V|^2)$$

- Para todos os menores caminhos entre os pares de vértices

- Estratégia “Gulosa” $\Theta(|V|^3 \ln |V|)$

Considerações sobre algoritmo de Dijkstra

- Para menor caminho entre dois vértices

$$\Theta(|V|^2)$$

- Para todos os menores caminhos entre os pares de vértices

$$\Theta(|V|^3 \ln |V|)$$

- É possível encontrar algoritmo mais eficiente?

Todos os menores caminhos entre pares de vértices

- Algoritmo Floyd-Warshall
- O algoritmo Floyd-Warshall foi publicado por Robert Floyd em 1962. Este algoritmo é o mesmo que foi publicado por Bernard Roy em 1959 e também por Stephen Warshall em 1962 para determinar o fechamento transitivo de um grafo. O formato atual do algoritmo de Floyd-Warshall com três laços de repetição foi descrito por Peter Ingberman em 1962.

https://pt.wikipedia.org/wiki/Algoritmo_de_Floyd-Warshall

Todos os menores caminhos entre pares de vértices

- Algoritmo Floyd-Warshall (Recebendo grafo com N vértices)

1 Floyd-Warshall(w, n) // w: pesos, n: número de vertices

2 {

3 for i = 1 to n do // inicializar, $D(0) = [w_{ij}]$

4 for j = 1 to n do

5 {

6 $d[i, j] = w[i, j];$

7 }

8 for k = 1 to n do // Computar $D(k)$ de $D(k-1)$

9 for i = 1 to n do

10 for j = 1 to n do

11 if ($d[i, k] + d[k, j] < d[i, j]$)

12 {

13 $d[i, j] = d[i, k] + d[k, j];$

14 }

15 return $d[1..n, 1..n];$

16 }

Exemplo: Algoritmo Floyd-Warshall

```
double d[num_vertices][num_vertices];

// Inicializar a matriz d:  Theta(|V|^2)
// ...

// Run Floyd-Warshall
for ( int k = 0; k < num_vertices; ++k ) {
    for ( int i = 0; i < num_vertices; ++i ) {
        for ( int j = 0; j < num_vertices; ++j ) {
            d[i][j] = std::min( d[i][j], d[i][k] + d[k][j] );
        }
    }
}
```

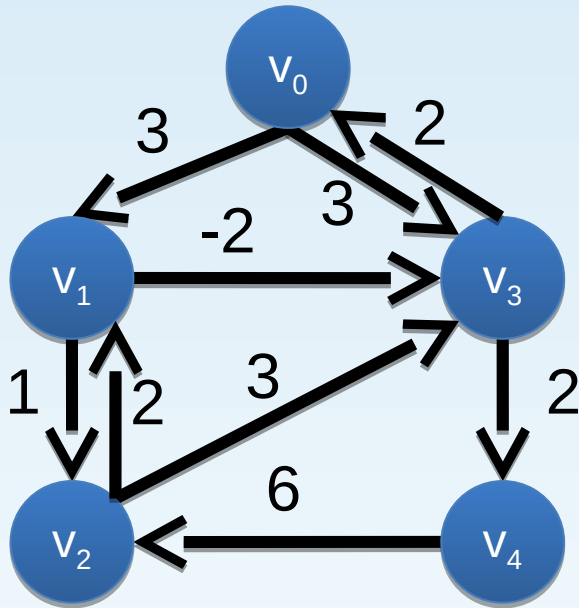
Exemplo: Algoritmo Floyd-Warshall

```
Floyd_Warshall(int n, int W[1..n, 1..n]) {  
    array d[1..n, 1..n]  
    for i = 1 to n do {  
        for j = 1 to n do {  
            d[i,j] = W[i,j]  
            pred[i,j] = null  
        }  
    }  
    for k = 1 to n do  
        for i = 1 to n do  
            for j = 1 to n do  
                if (d[i,k] + d[k,j]) < d[i,j]) {  
                    d[i,j] = d[i,k] + d[k,j]  
                    pred[i,j] = k  
                }  
            }  
        }  
    return d  
}
```

Considerações sobre algoritmo Floyd-Warshall

- Algoritmo Floyd-Warshall $\Theta(|V|^3)$
- Armazena todos os menores caminhos.
- Estratégia “Programação dinâmica”

Exemplo: Algoritmo Floyd-Warshall

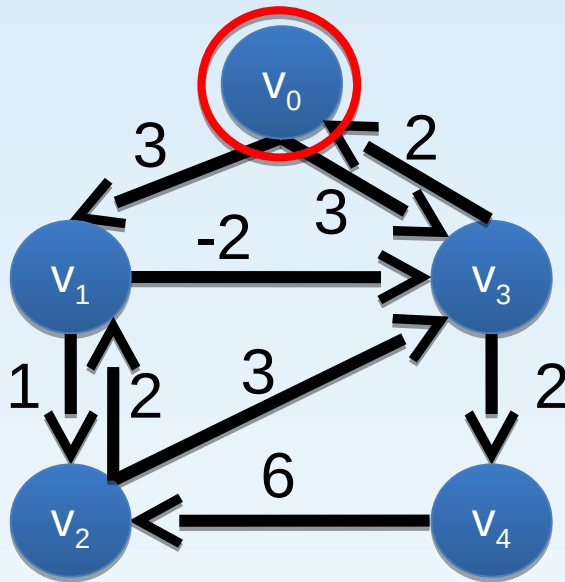


Seja $d_k(s,t)$ = comprimento mínimo entre s - t usando somente $\{s,t,v_0,\dots,v_{k-1}\}$.

$d_{k+1}(s,t)$ = mínimo entre

$$d_k(s,t), \\ d_k(s,v_k) + d_k(v_k,t)$$

Exemplo: Algoritmo Floyd-Warshall



$d_1(v_0, v_1) =$
 mínimo entre
 $d_0(v_0, v_1),$
 $d_0(v_0, v_0) + d_0(v_0, v_1)$

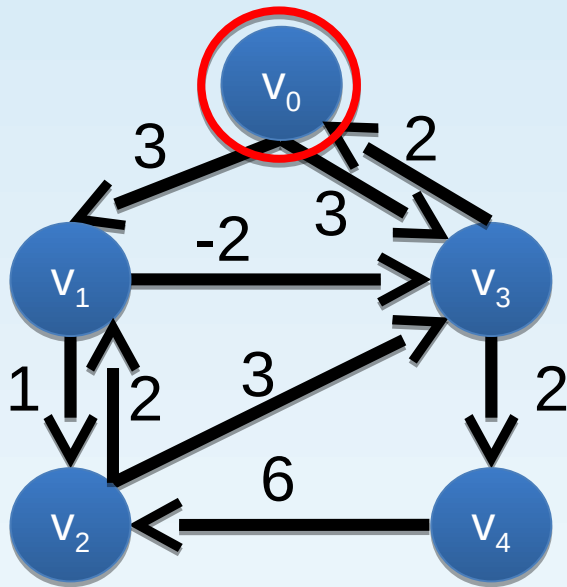
d_0

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

d_1

0				
	0			
		0		
			0	
				0

Exemplo: Algoritmo Floyd-Warshall



$d_1(v_0, v_2) =$
 mínimo entre
 $d_0(v_0, v_2),$
 $d_0(v_0, v_0) + d_0(v_0, v_2)$

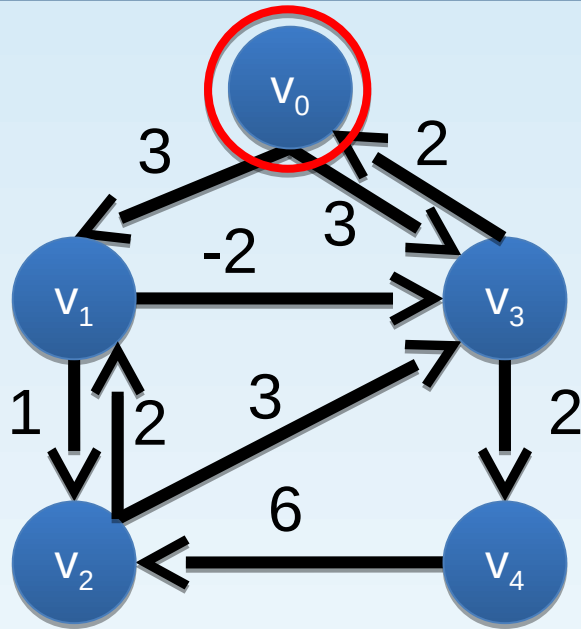
d_0

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

d_1

0	3			
	0			
		0		
			0	
				0

Exemplo: Algoritmo Floyd-Warshall



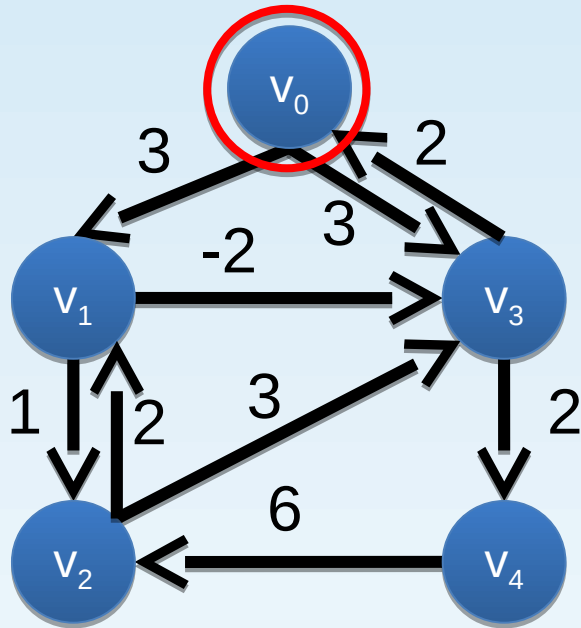
$d_1(v_0, v_3) =$
 mínimo entre
 $d_0(v_0, v_3),$
 $d_0(v_0, v_0) + d_0(v_0, v_3)$

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

d_1

0	3	∞		
	0			
		0		
			0	
				0

Exemplo: Algoritmo Floyd-Warshall



d_0

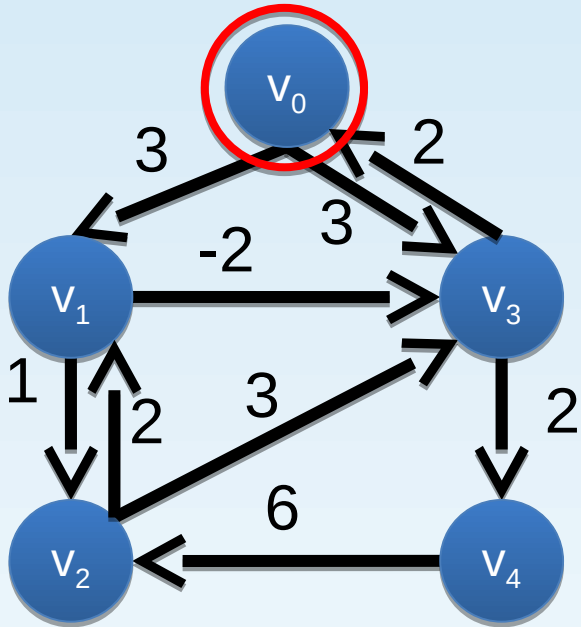
0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

$d_1(v_0, v_4) =$
 mínimo entre
 $d_0(v_0, v_4),$
 $d_0(v_0, v_0) + d_0(v_0, v_4)$

d_1

0	3	∞	3	
	0			
		0		
			0	
				0

Exemplo: Algoritmo Floyd-Warshall



$d_1(v_1, v_0) =$
 mínimo entre
 $d_0(v_1, v_0),$
 $d_0(v_1, v_0) + d_0(v_0, v_0)$

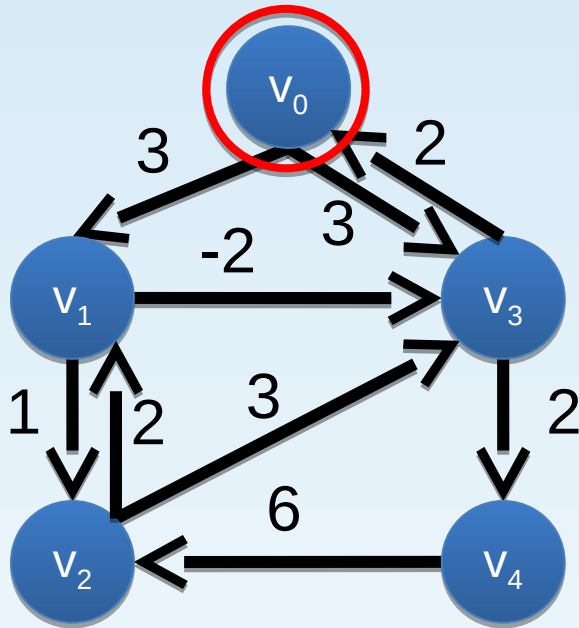
d_0

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

d_1

0	3	∞	3	∞
	0			
		0		
			0	
				0

Exemplo: Algoritmo Floyd-Warshall



$d_1(v_1, v_2) =$
 mínimo entre
 $d_0(v_1, v_2),$
 $d_0(v_1, v_0) + d_0(v_0, v_2)$

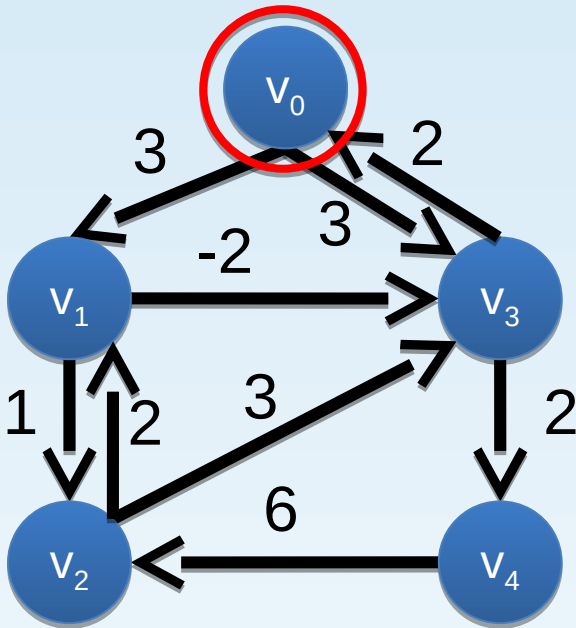
d_0

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

d_1

0	3	∞	3	∞
∞	0			
		0		
			0	
				0

Exemplo: Algoritmo Floyd-Warshall



$d_1(v_1, v_3) =$
 mínimo entre
 $d_0(v_1, v_3),$
 $d_0(v_1, v_0) + d_0(v_0, v_3)$

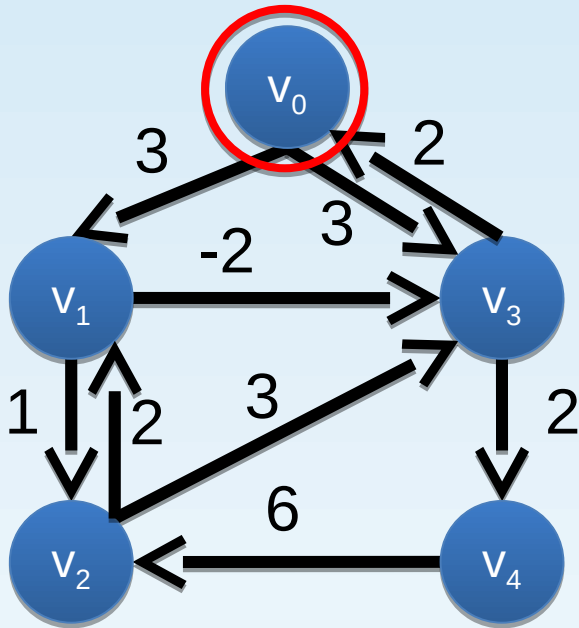
d_0

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

d_1

0	3	∞	3	∞
∞	0	1		
		0		
			0	
				0

Exemplo: Algoritmo Floyd-Warshall



$d_1(v_1, v_4) =$
 mínimo entre
 $d_0(v_1, v_4),$
 $d_0(v_1, v_0) + d_0(v_0, v_4)$

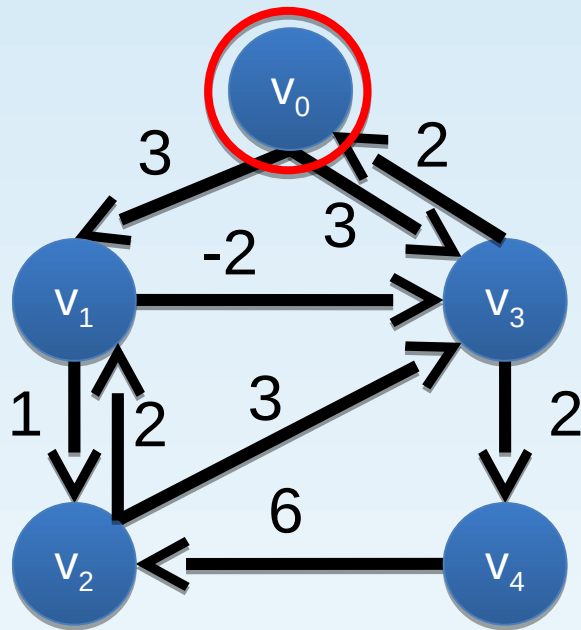
d_0

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

d_1

0	3	∞	3	∞
∞	0	1	-2	
		0		
			0	
				0

Exemplo: Algoritmo Floyd-Warshall



d_0

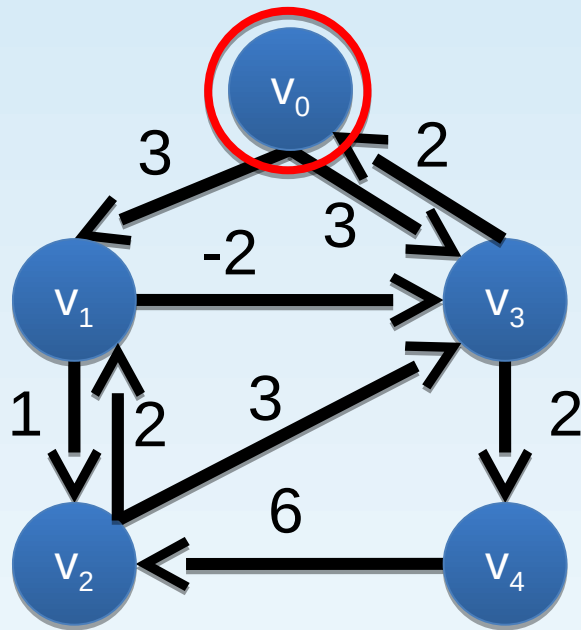
0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

d_1

0	3	∞	3	∞
∞	0	1	-2	∞
		0		
			0	
				0

$d_1(v_2, v_0) =$
 mínimo entre
 $d_0(v_2, v_0),$
 $d_0(v_2, v_0) + d_0(v_0, v_0)$

Exemplo: Algoritmo Floyd-Warshall



d_0

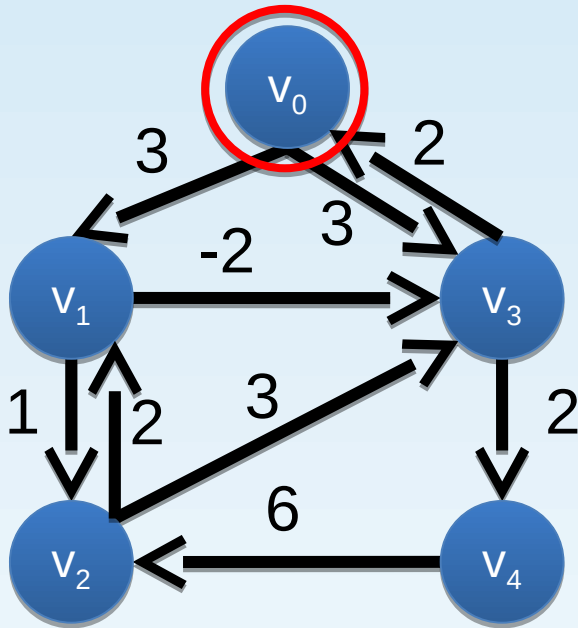
0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

d_1

0	3	∞	3	∞
∞	0	1	-2	∞
∞		0		
			0	
				0

$d_1(v_2, v_1) =$
 mínimo entre
 $d_0(v_2, v_1),$
 $d_0(v_2, v_0) + d_0(v_0, v_1)$

Exemplo: Algoritmo Floyd-Warshall



d_0

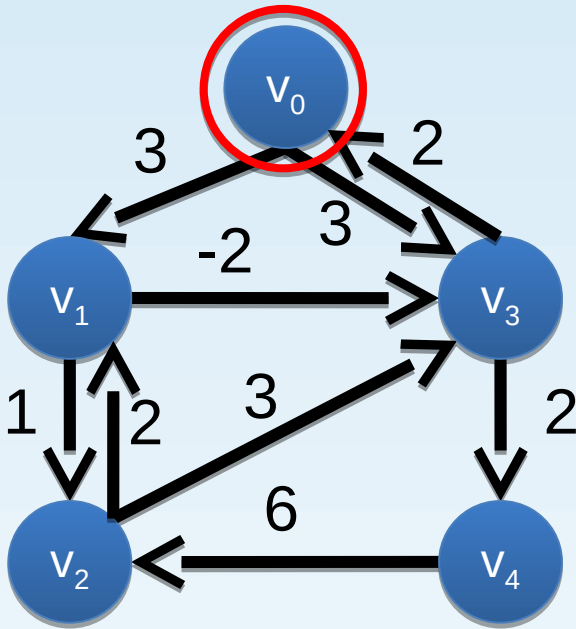
0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

d_1

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0		
			0	
				0

$d_1(v_2, v_3) =$
 mínimo entre
 $d_0(v_2, v_3),$
 $d_0(v_2, v_0) + d_0(v_0, v_3)$

Exemplo: Algoritmo Floyd-Warshall



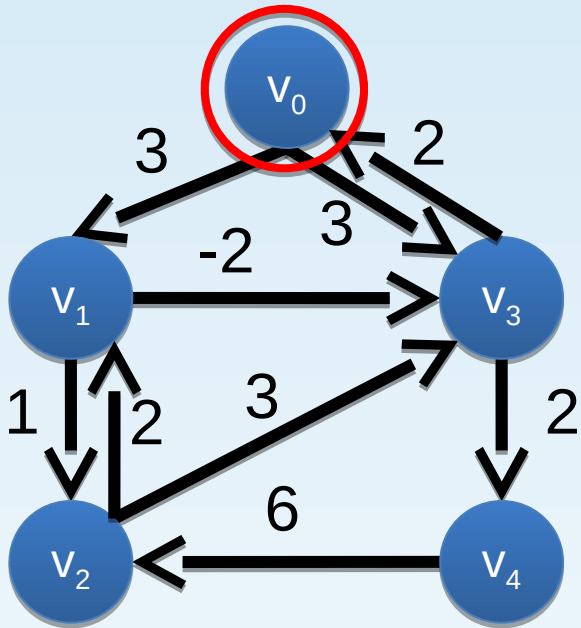
0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

$d_1(v_2, v_4) =$
 mínimo entre
 $d_0(v_2, v_4),$
 $d_0(v_2, v_0) + d_0(v_0, v_4)$

d_1

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	
			0	
				0

Exemplo: Algoritmo Floyd-Warshall



d_0

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

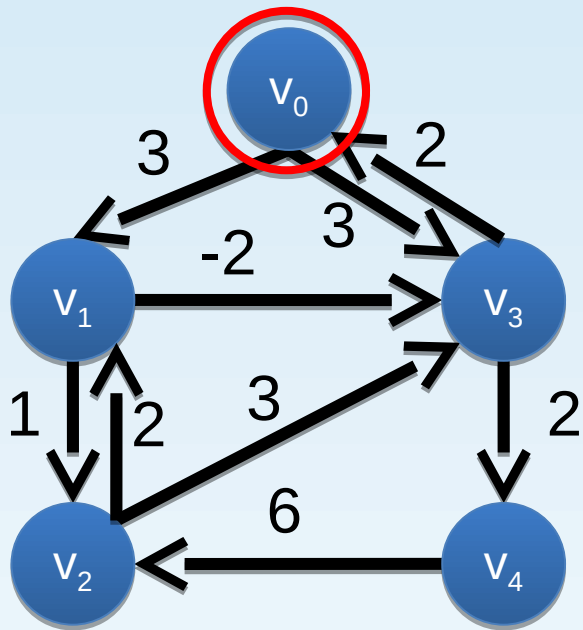
d_1

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
			0	
				0

$d_1(v_3, v_0) =$
 mínimo entre
 $d_0(v_3, v_0),$
 $d_0(v_3, v_0) + d_0(v_0, v_0)$



Exemplo: Algoritmo Floyd-Warshall



$d_1(v_3, v_1) =$
 mínimo entre
 $d_0(v_3, v_1),$
 $d_0(v_3, v_0) + d_0(v_0, v_1)$

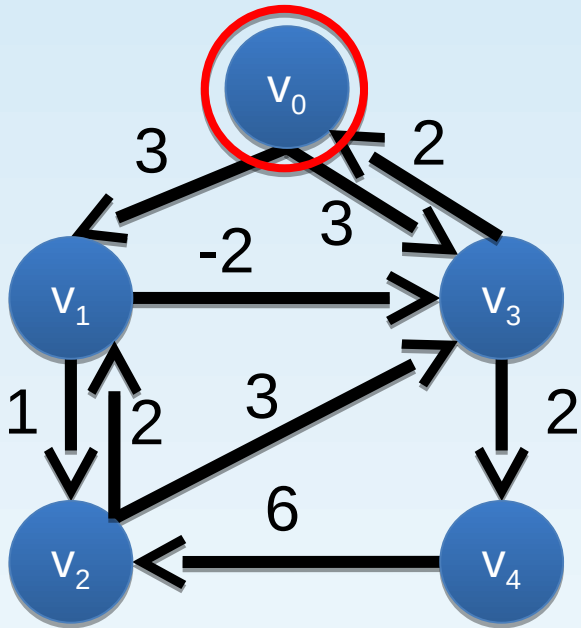
d_0

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

d_1

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2			0	
				0

Exemplo: Algoritmo Floyd-Warshall



d_0

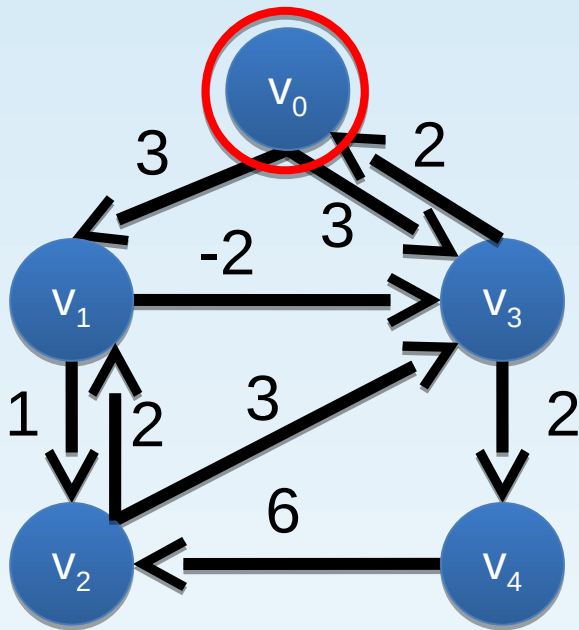
0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

d_1

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	5		0	
				0

$d_1(v_3, v_2) =$
 mínimo entre
 $d_0(v_3, v_2),$
 $d_0(v_3, v_0) + d_0(v_0, v_2)$

Exemplo: Algoritmo Floyd-Warshall



d_0

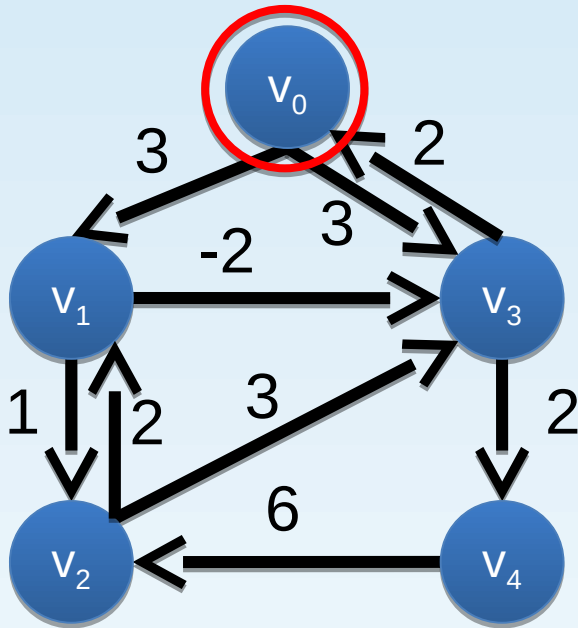
0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

d_1

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	5	∞	0	
				0

$d_1(v_3, v_4) =$
 mínimo entre
 $d_0(v_3, v_4),$
 $d_0(v_3, v_0) + d_0(v_0, v_4)$

Exemplo: Algoritmo Floyd-Warshall



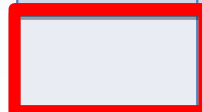
d_0

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

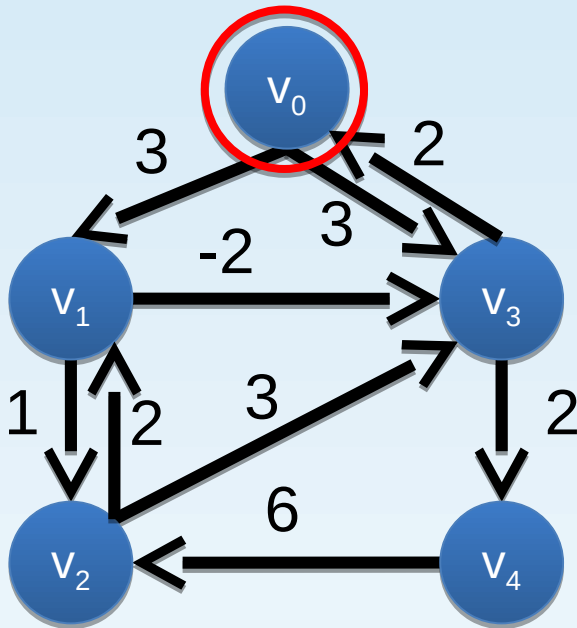
d_1

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	5	∞	0	2
				0

$d_1(v_4, v_0) =$
 mínimo entre
 $d_0(v_4, v_0),$
 $d_0(v_4, v_0) + d_0(v_0, v_0)$



Exemplo: Algoritmo Floyd-Warshall



$d_1(v_4, v_1) =$
 mínimo entre
 $d_0(v_4, v_1),$
 $d_0(v_4, v_0) + d_0(v_0, v_1)$

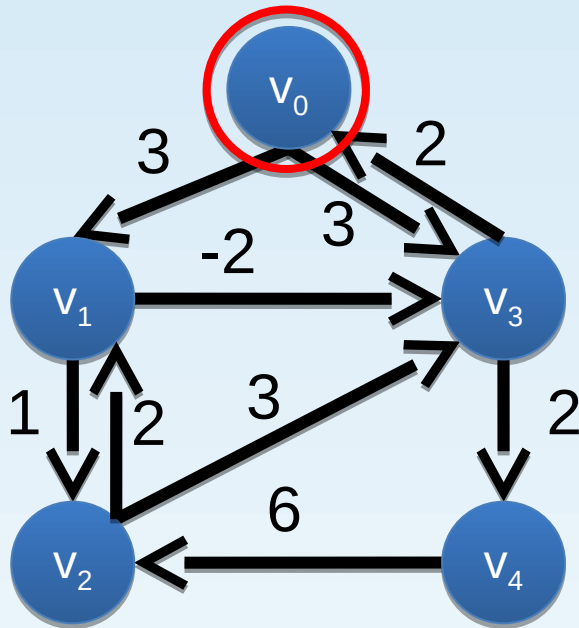
d_0

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

d_1

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	5	∞	0	2
∞				0

Exemplo: Algoritmo Floyd-Warshall



d_0

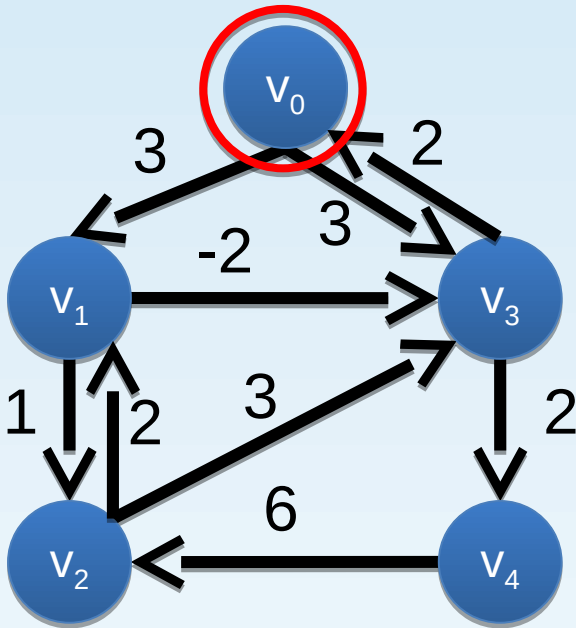
0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

d_1

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	5	∞	0	2
∞	∞			0

$d_1(v_4, v_2) =$
 mínimo entre
 $d_0(v_4, v_2),$
 $d_0(v_4, v_0) + d_0(v_0, v_2)$

Exemplo: Algoritmo Floyd-Warshall



$d_1(v_4, v_3) =$
 mínimo entre
 $d_0(v_4, v_3),$
 $d_0(v_4, v_0) + d_0(v_0, v_3)$

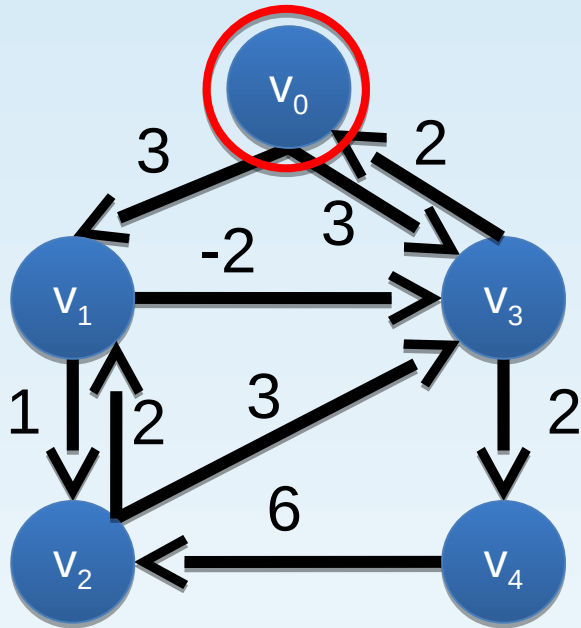
d_0

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

d_1

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	5	∞	0	2
∞	∞	6		0

Exemplo: Algoritmo Floyd-Warshall



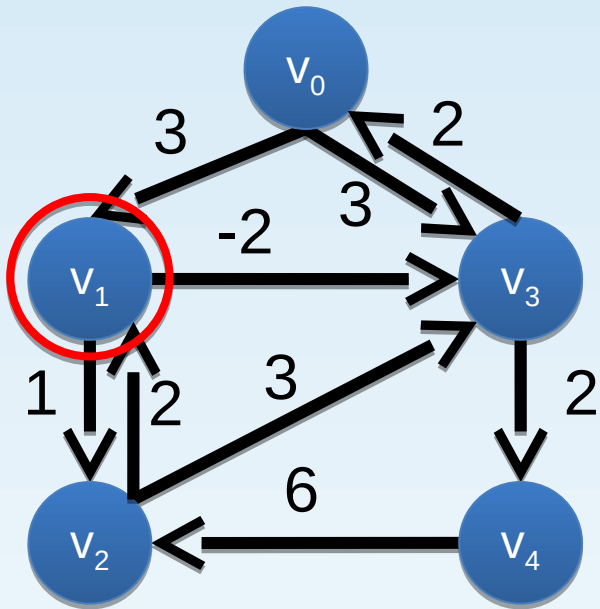
d_0

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	∞	∞	0	2
∞	∞	6	∞	0

d_1

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	5	∞	0	2
∞	∞	6	∞	0

Exemplo: Algoritmo Floyd-Warshall



d_1

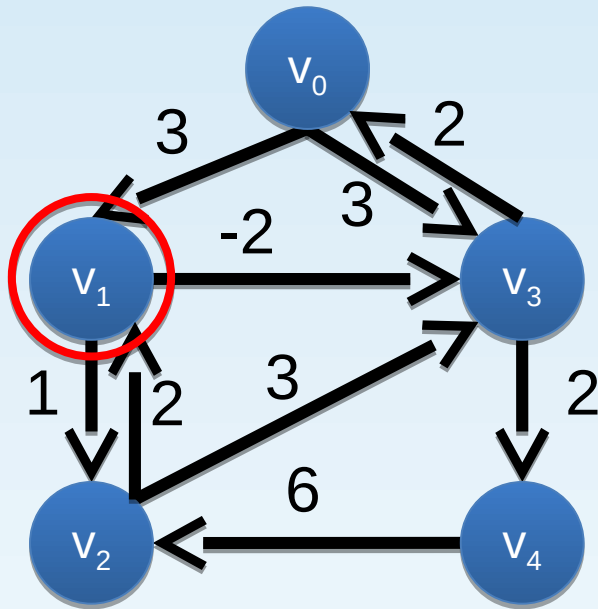
0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	5	∞	0	2
∞	∞	6	∞	0

d_2

0				
	0			
		0		
			0	
				0

$d_2(v_0, v_1) =$
 mínimo entre
 $d_1(v_0, v_1),$
 $d_1(v_0, v_1) + d_1(v_1, v_0)$

Exemplo: Algoritmo Floyd-Warshall



d_1

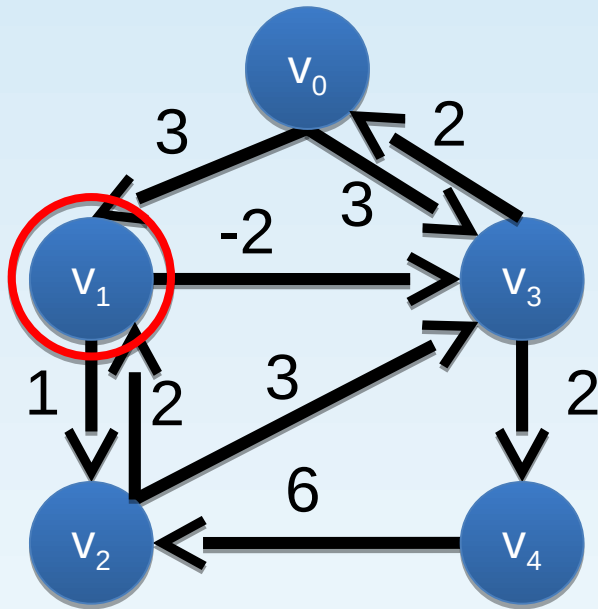
0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	5	∞	0	2
∞	∞	6	∞	0

d_2

0	3			
	0			
		0		
			0	
				0

$d_2(v_0, v_2) =$
 mínimo entre
 $d_1(v_0, v_2),$
 $d_1(v_0, v_1) + d_1(v_1, v_2)$

Exemplo: Algoritmo Floyd-Warshall



d_1

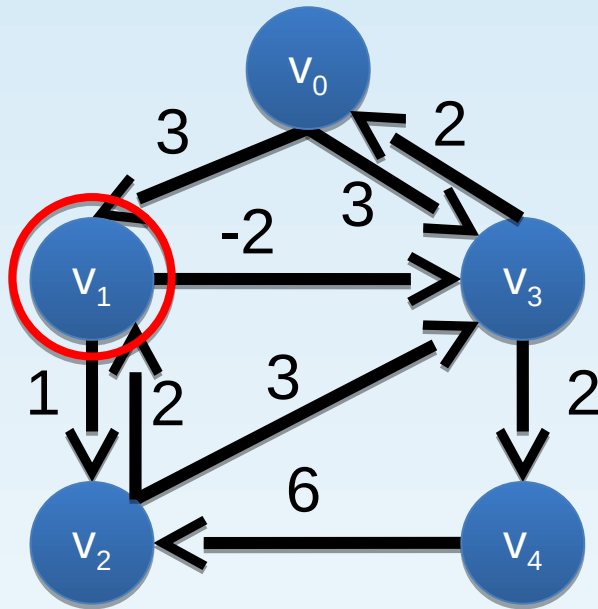
0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	5	∞	0	2
∞	∞	6	∞	0

d_2

0	3	4		
	0			
		0		
			0	
				0

$d_2(v_0, v_3) =$
 mínimo entre
 $d_1(v_0, v_3),$
 $d_1(v_0, v_1) + d_1(v_1, v_3)$

Exemplo: Algoritmo Floyd-Warshall



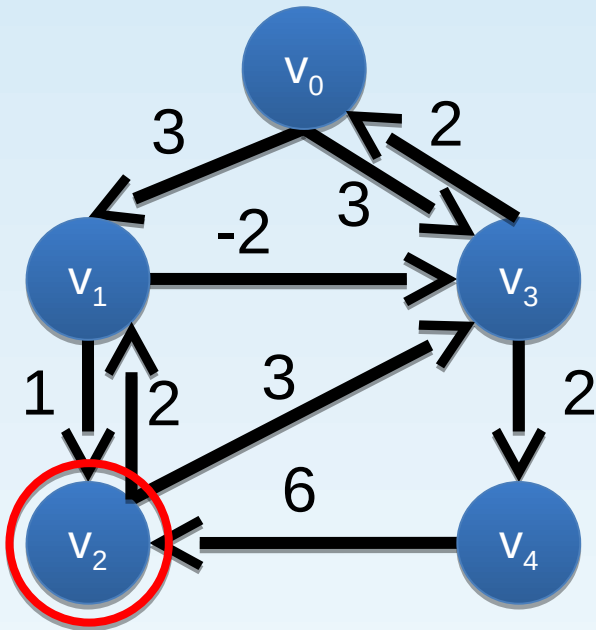
d_1

0	3	∞	3	∞
∞	0	1	-2	∞
∞	2	0	3	∞
2	5	∞	0	2
∞	∞	6	∞	0

d_2

0	3	4	1	∞
∞	0	1	-2	∞
∞	2	0	0	∞
2	5	6	0	2
∞	∞	6	∞	0

Exemplo: Algoritmo Floyd-Warshall



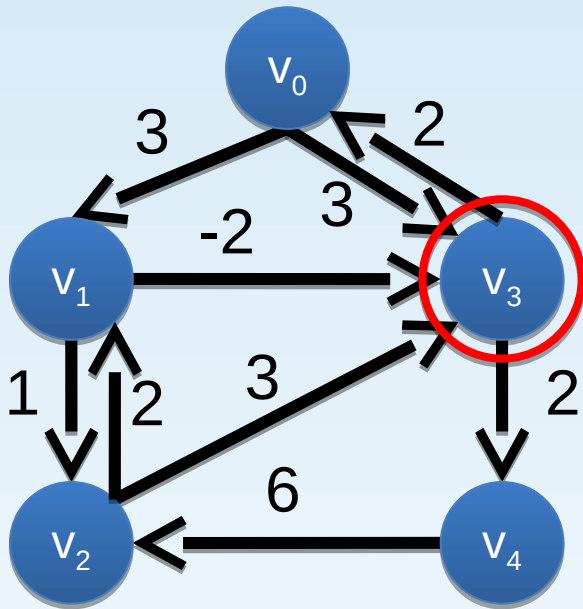
d_2

0	3	4	1	∞
∞	0	1	-2	∞
∞	2	0	0	∞
2	5	6	0	2
∞	∞	6	∞	0

d_3

0	3	4	1	∞
∞	0	1	-2	∞
∞	2	0	0	∞
2	5	6	0	2
∞	8	6	6	0

Exemplo: Algoritmo Floyd-Warshall



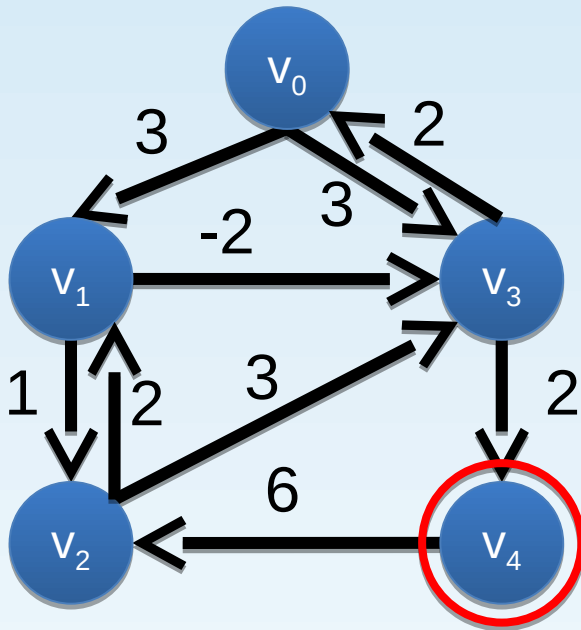
d_3

0	3	4	1	∞
∞	0	1	-2	∞
∞	2	0	0	∞
2	5	6	0	2
∞	8	6	6	0

d_4

0	3	4	1	3
0	0	1	-2	0
2	2	0	0	2
2	5	6	0	2
8	8	6	6	0

Exemplo: Algoritmo Floyd-Warshall



d_4

0	3	4	1	3
0	0	1	-2	0
2	2	0	0	2
2	5	6	0	2
8	8	6	6	0

d_5

0	3	4	1	3
0	0	1	-2	0
2	2	0	0	2
2	5	6	0	2
8	8	6	6	0

(Pseudocódigo): Algoritmo Floyd-Warshall

$K = 0$, $d_k(i,j) = l(i,j)$ if $(i,j) \in A$, caso contrário ∞

While ($k < n$)

For cada $i \in V$

For cada $j \in V$

$d_{k+1}(i,j) = \min \{ d_k(i,j), d_k(i, v_{k-1}) + d_k(v_{k-1}, j) \}$

End

End

$k = k + 1$

End

Percursos de Euler

- Cerca de 1736 dc, Leonhard Euler descreveu um problema, o qual chamou de “geometria da posição”. O problema envolvia as pontes da cidade de Königsberg (Prússia), hoje Kaliningrado (Rússia).

Pontes de Königsberg

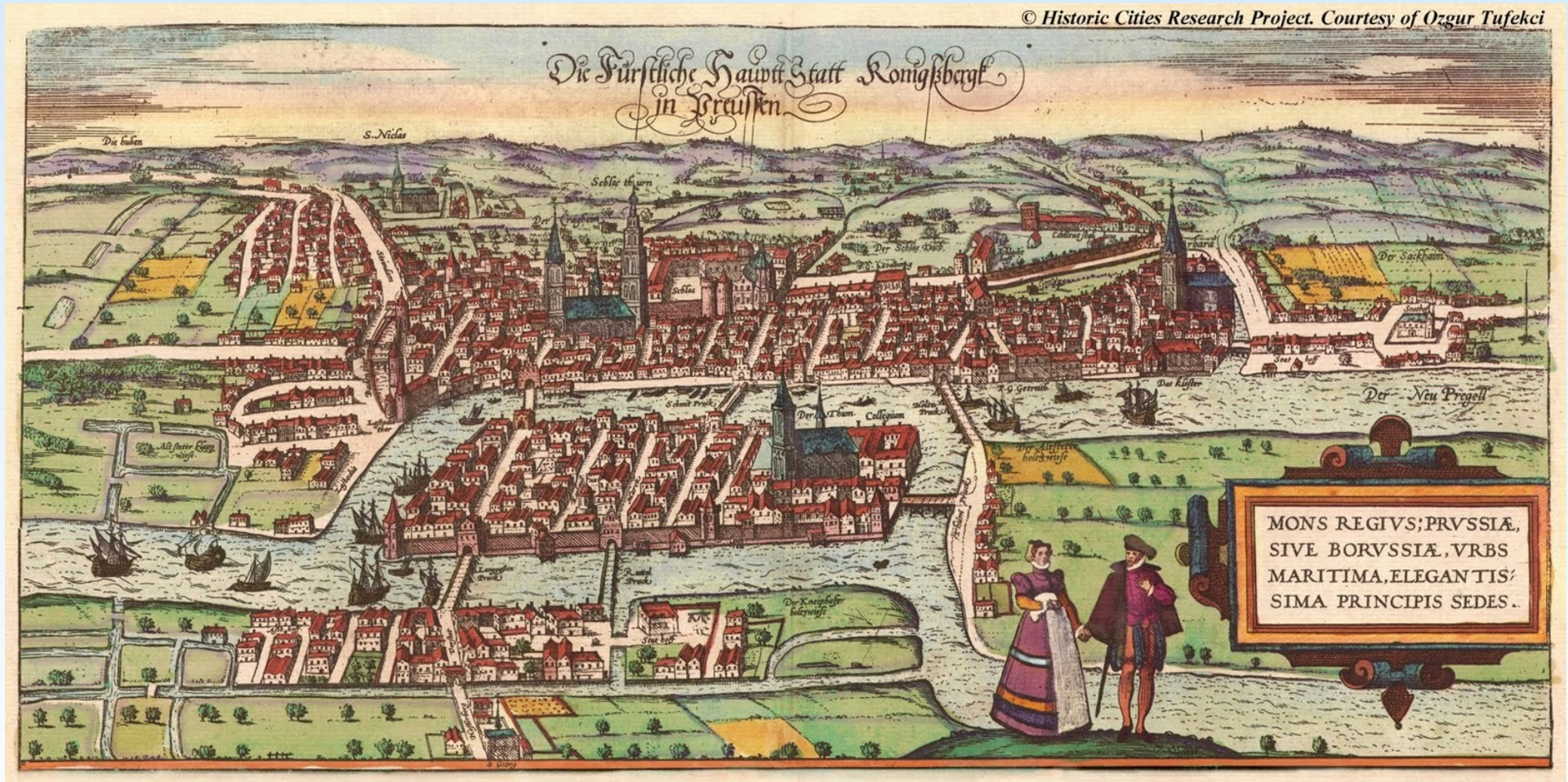


Imagem: autoentusiastascClassic.com.br

Pontes de Königsberg

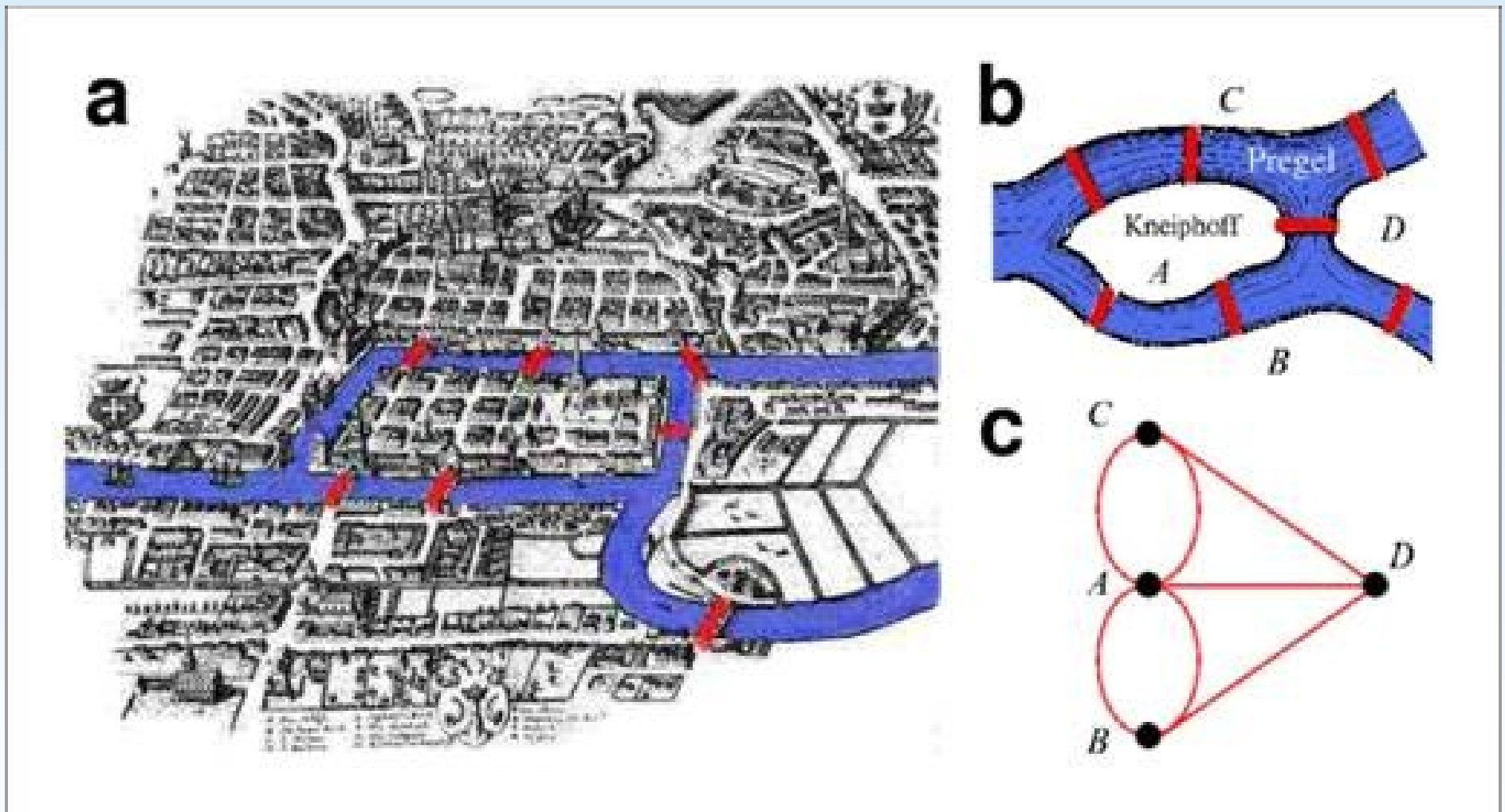


Imagem: www.emaze.com

Percursos de Euler

- Cerca de 1736 dc, Leonhard Euler descreveu um problema, o qual chamou de “geometria da posição”. O problema envolvia as pontes da cidade de Königsberg (Prússia), hoje Kaliningrado (Rússia).
- Seria possível atravessar todas as pontes sem repetir nenhuma?

Percursos de Euler

- Cerca de 1736 dc, Leonhard Euler descreveu um problema, o qual chamou de “geometria da posição”. O problema envolvia as pontes da cidade de Königsberg (Prússia), hoje Kaliningrado (Rússia).
- Seria possível atravessar todas as pontes sem repetir nenhuma? **Não.**

Percursos de Euler

- **Circuito/Ciclo de Euler** é aquele que: inicia e termina no mesmo vértice; inclui cada vértice pelo menos uma vez, e cada aresta somente uma vez.
- **Trilha de Euler** inicia e termina em vértices diferentes, mas inclui cada vértice pelo menos uma vez, e cada aresta somente uma vez.

Condições para existência de Circuito de Euler

- Grafo deve ser conectado.

Condições para existência de Circuito de Euler

- Grafo deve ser conectado.
- Todo vértice deve ter grau par.

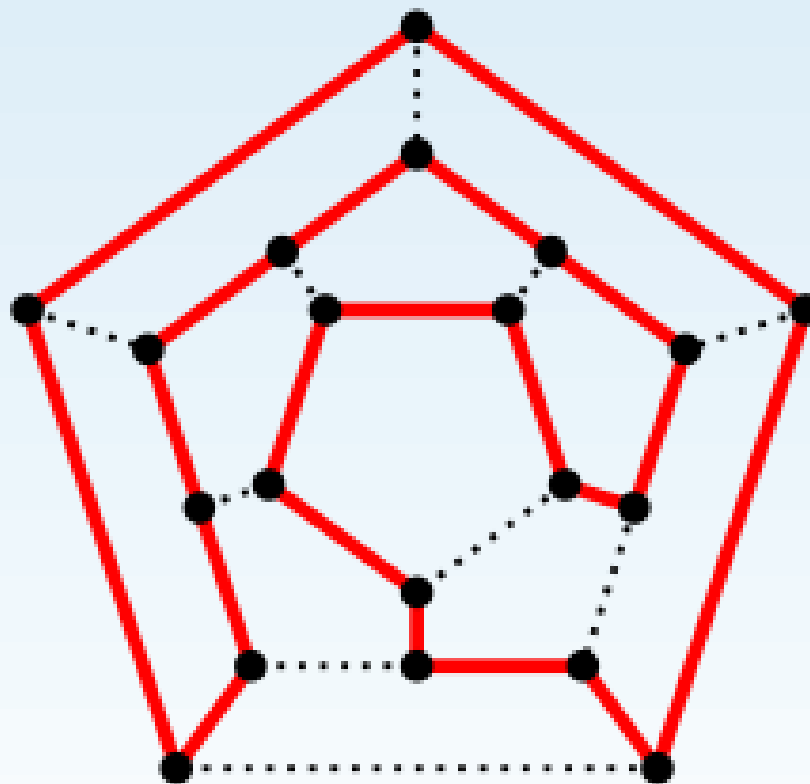
Condições para existência de Circuito de Euler

- Grafo deve ser conectado.
- Todo vértice deve ter grau par.
- Um grafo com circuito de Euler é chamado de **Grafo Euleriano**.

Percursos de Hamilton

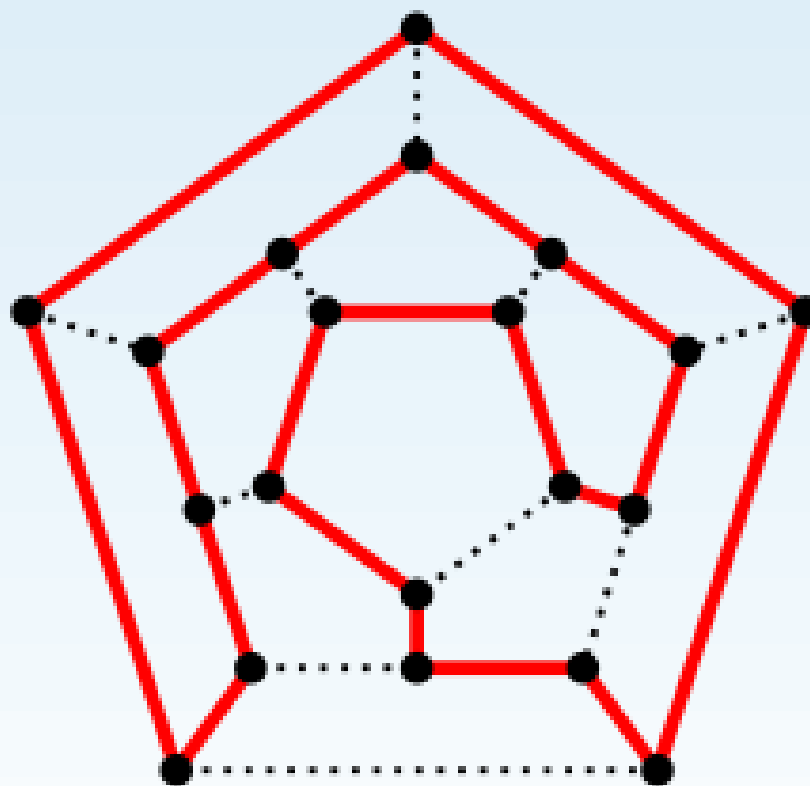
- William Rowan Hamilton descreveu um jogo, onde cada vértice de um grafo especial (dodecaedro) deve ser visitado somente uma vez (no caso de circuito, início e fim considerados como um só).
- Hamilton, William Rowan (1858), "Account of the Icosian Calculus", *Proceedings of the Royal Irish Academy*, 6: 415–416.

Percursos de Hamilton



en.wikipedia.org

Percursos de Hamilton



Circuito/Ciclo de
Hamilton

en.wikipedia.org

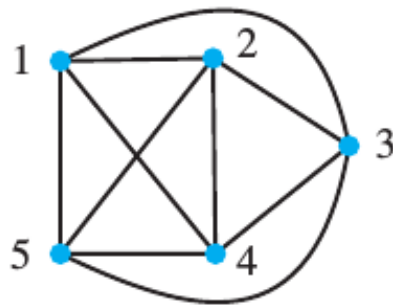
Condições para existência de Circuitos de Hamilton

- Teorema de Ore
- Se G é um grafo simples, com v vértices, sendo $v \geq 3$, e para cada dois x e y vértices não adjacentes a soma dos graus de x e y for no mínimo v , então G terá um ciclo Hamiltoniano.

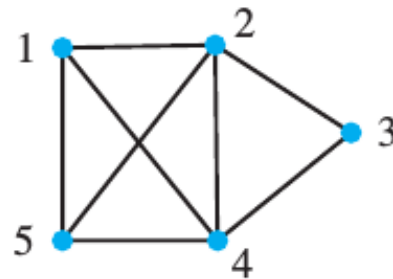
Condições para existência de Circuitos de Hamilton

- Teorema de Dirac
- Se G é um grafo simples, com v vértices, sendo $v \geq 3$, e todo vértice ter pelo menos grau $v/2$, então G terá um ciclo Hamiltoniano.

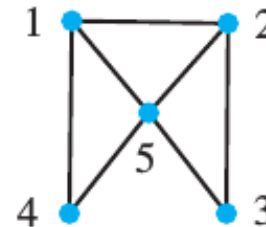
Exercício: Existem Circuitos Eulerianos nos seguintes grafos? Quais?



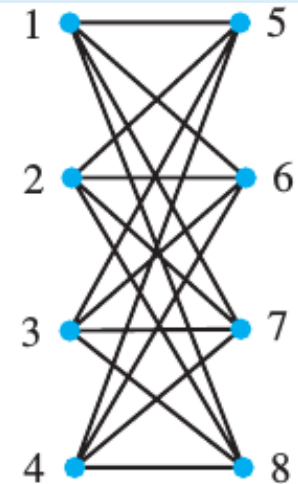
a



b



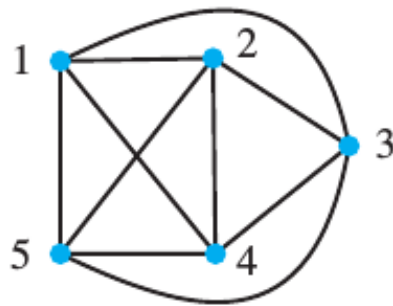
c



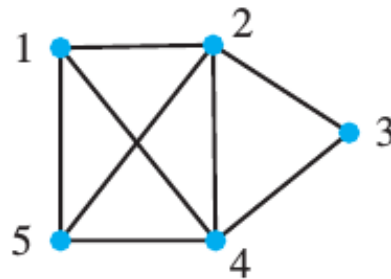
d

Bogart, Stein & Drysdale, 2006.

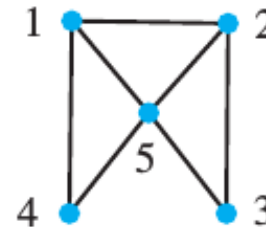
Exercício: Existem Circuitos Eulerianos nos seguintes grafos? Quais?



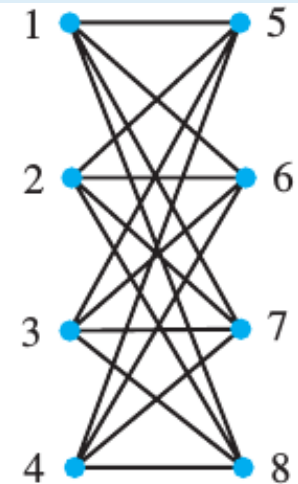
a



b



c

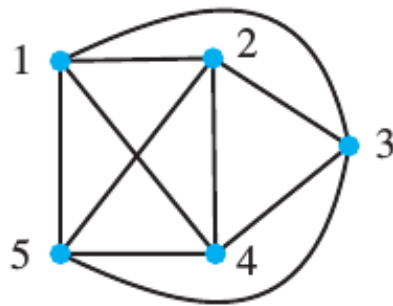


d

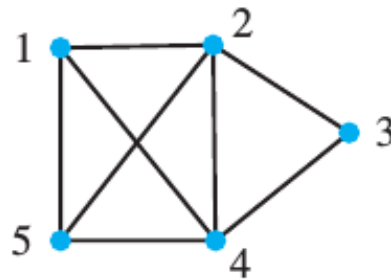
a: 1, 2, 3, 4, 5, 1, 4, 2, 5, 3, 1

Bogart, Stein & Drysdale, 2006.

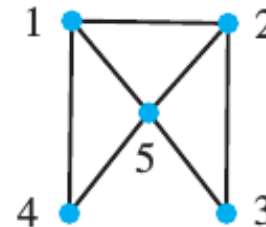
Exercício: Existem Circuitos Eulerianos nos seguintes grafos? Quais?



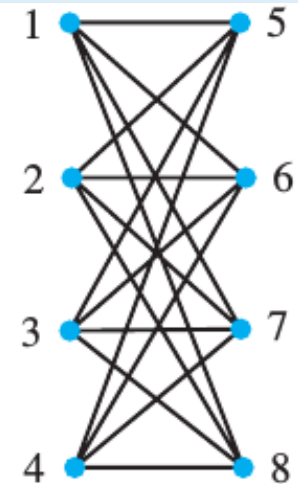
a



b



c



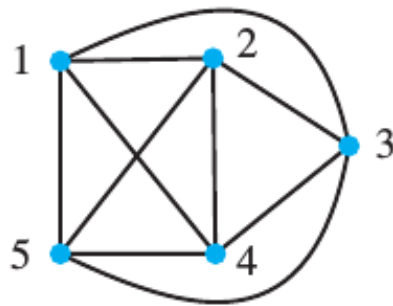
d

a: 1,2,3,4,5,1,4,2,5,3,1

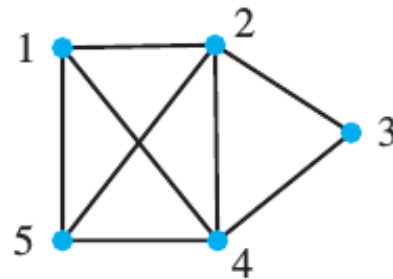
b: não

Bogart, Stein & Drysdale, 2006.

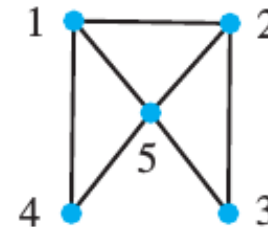
Exercício: Existem Circuitos Eulerianos nos seguintes grafos? Quais?



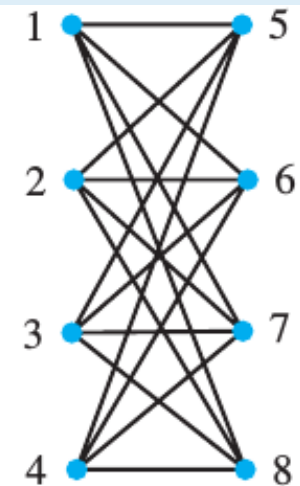
a



b



c



d

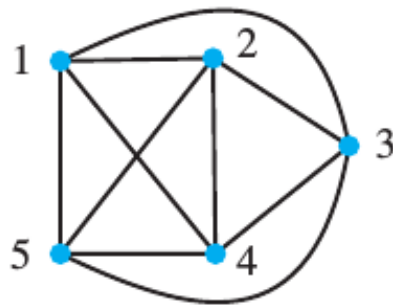
a: 1,2,3,4,5,1,4,2,5,3,1

b: não

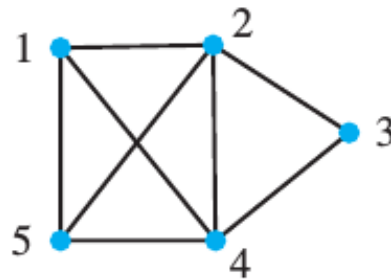
c: não

Bogart, Stein & Drysdale, 2006.

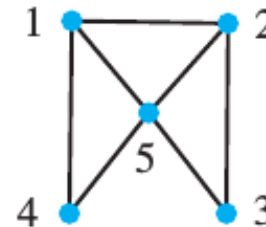
Exercício: Existem Circuitos Eulerianos nos seguintes grafos? Quais?



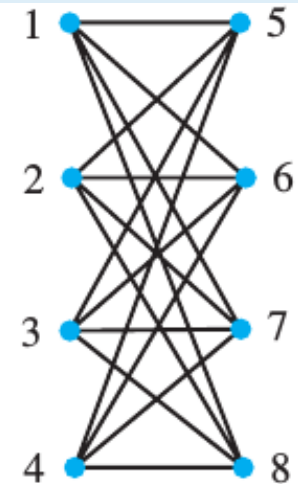
a



b



c



d

a: 1,2,3,4,5,1,4,2,5,3,1

b: não

c: não

d: 1,5,2,6,3,7,4,8,1,6,4,5,3,8,2,7,1

Bogart, Stein & Drysdale, 2006.

Exercício: Existem Circuitos Hamiltonianos nos seguintes grafos? Quais?



a



b



c



d

Quais hipóteses de Ore e/ou Dirac são satisfeitas?

Bogart, Stein & Drysdale, 2006.

Referências Bibliográficas

- Bogart, K. & Stein, C. & Drysdale, R. Discrete Mathematics for Computer Science. Key College Pub., 2006.
- Bondy, J. & Murty, *Graph Theory with Applications*, Elsevier, 1982.
- Ruohonen, K. *Graph Theory*, 2013.
- Sedgewick, R. & Wayne, K. *Algorithms* (4th ed.), Addison-Wesley, 2011.
- Wilson, R. & Watkins, J. *Graphs: An Introductory Approach*. John Wiley, 1990.
- Ziviani, N. *Projeto de algoritmos com implementações em pascal e C*. Cengage, 2011.